

基于 YOLO 的车牌号识别系统学习文档

一. 环境搭建:

1.从哔哩哔哩搜索yolo11安装视频及安装文档，进行认真学习，并按照视频进行了安装，由于初学，安装均不成功，通过多次摸索，最终主要参考以下链接完成：

<https://www.bilibili.com/opus/1021487181719404550>，YOLOv11(Ultralytics)环境配置，适合0基础纯小白，手把手教 - 哔哩哔哩

Anaconda3文件包下载网站：<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>

Pycharm安装包下载

<https://www.jetbrains.com/zh-cn/pycharm/download/?section=windows>

两个文件选择合适版本进行安装。中间由于版本问题，进行了多次下载安装。

2.创建yolo11环境

由于我所使用的电脑为Intel核显，没有独立显卡无法使用GPU进行训练，需用CPU进行训练。

打开anaconda prompt，运行以下命令创建yolo11环境：

```
conda create -n yolov11 python=3.10
```

创建完成后，输入

```
conda activate yolov11
```

进入yolo环境。

3. 安装pytorch

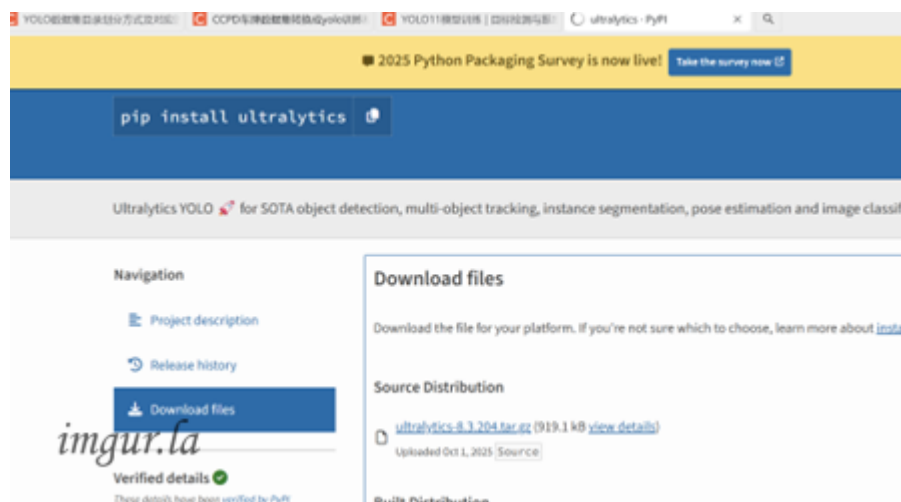
```
conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

4. 安装ultralytics库

```
pip install ultralytics
```

5.下载YOLOv11 (ultralytics) 源码,

由于哔哩哔哩搜索的学习文档中给出的网址一直无法登录，重新学习实验室下发的链接中文档，找到下载链接，成功下载ultralytics源码见下图。



下载预训练权重文件yolo11n、yolo11s、yolo11m，下载界面如下：

A screenshot of the YOLOv11 models page. It features a table with performance metrics for various models. The table has five columns: Model, size (pixels), mAPval 50-95, Speed CPU ONNX (ms), and Speed T4 TensorRT (ms). The models listed are YOLO11n, YOLO11s, YOLO11m, YOLO11l, and YOLO11x. Above the table, there's a section titled 'Detection (COCO)' with links to 'Detection Docs' and 'COCO dataset'.

Model	size (pixels)	mAPval 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT (ms)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2

6. pycharm导入环境

导入界面：



二. 下载和标注数据集

1.按照文档内容，下载数据集。

网址: <https://pan.baidu.com/s/1-WmsMt7Zzx3jmLHVEYpdaw>

提取码: 2111

2.学习文档中ccpd数据集处理，并从哔哩哔哩搜索LabelStudio安装教程及数据标记教程。

参考文档及视频:

https://blog.csdn.net/m0_72915515/article/details/134248502#:~:text=%E9%87%87%E7%94%A8Anaconda%E5%88%9B%E5%BB%BA%E5%90%8D%E4%B8%BAlabel_studio%E7%9A%84%E8%99%9A%E6%8B%9F%E7%8E%AF%E5%A2%83 , 标注工具——Label Studio安装与简单使用-CSDN博客

https://www.bilibili.com/video/BV1Ht4y1d7Yr/?spm_id_from=333.337.search-card.all.click&vd_source=0e4eaed4406fd009f7a2bd20660a23a4 , 数据标注的方法 制作自己的数据 labelstudio的使用_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1K3zwYQEgT/?spm_id_from=333.337.search-card.all.click&vd_source=0e4eaed4406fd009f7a2bd20660a23a4 , label studio使用教程_哔哩哔哩_bilibili

打开anaconda prompt, 运行以下命令:

```
pip install Label-Studio
```

安装成功后，运行Label-Studio start，进入web页面进行数据集标注。

3.在导入本地图片过程中，由于使用了中文翻译版，一直提示“要插入新节点的节点不是该节点的子节点”错误

查找多个文档未解决，最后重新登陆英文版界面，成功将数据导入，并完成标注，进行导出，一共完成113张车牌数据标注。

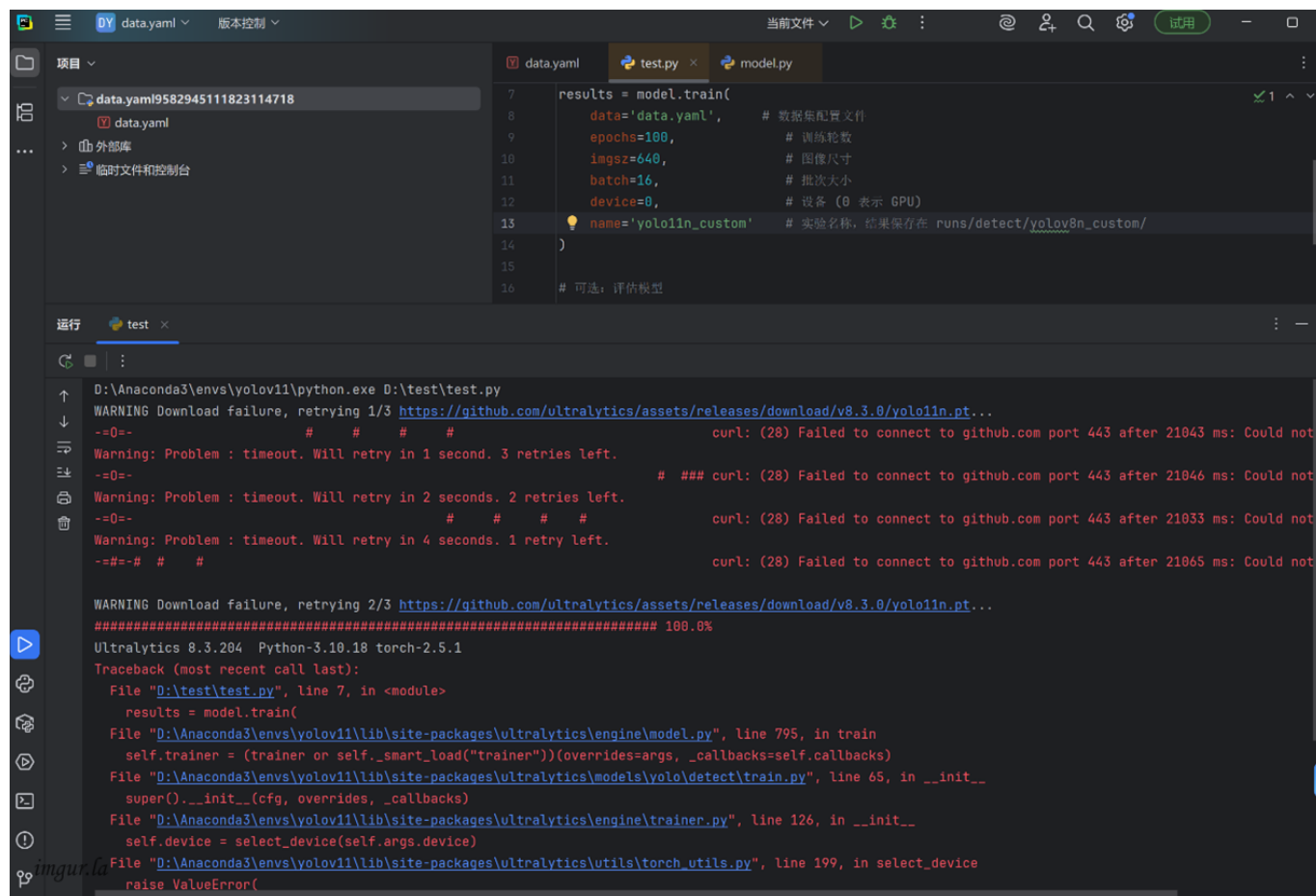
三、划分数据集，配置并训练模型

1.数据集划分：

根据文档，将数据集目录在划分为train、val、test三个目录。

2.data.yaml文档配置及模型训练

第一便训练提示如下错误，根据错误提示先进行网络配置排查，最终调整网络设置进行解决：



```
data.yaml
7 results = model.train(
8     data='data.yaml', # 数据集配置文件
9     epochs=100,        # 训练轮数
10    imgsz=640,         # 图像尺寸
11    batch=16,          # 批次大小
12    device=0,           # 设备 (0 表示 GPU)
13    name='yolo11n_custom' # 实验名称, 结果保存在 runs/detect/yolov8n_custom/
14)
15
16 # 可选: 评估模型

运行 test x
D:\Anaconda3\envs\yolov11\python.exe D:\test\test.py
WARNING Download failure, retrying 1/3 https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt...
--0-- # # # # curl: (28) Failed to connect to github.com port 443 after 21043 ms: Could not
Warning: Problem : timeout. Will retry in 1 second. 3 retries left.
--0-- # # # # curl: (28) Failed to connect to github.com port 443 after 21046 ms: Could not
Warning: Problem : timeout. Will retry in 2 seconds. 2 retries left.
--0-- # # # # curl: (28) Failed to connect to github.com port 443 after 21033 ms: Could not
Warning: Problem : timeout. Will retry in 4 seconds. 1 retry left.
--#--# # # # curl: (28) Failed to connect to github.com port 443 after 21065 ms: Could not

WARNING Download failure, retrying 2/3 https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt...
##### 100.0%
Ultralytics 8.3.204 Python-3.10.18 torch-2.5.1
Traceback (most recent call last):
  File "D:\test\test.py", line 7, in <module>
    results = model.train(
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\model.py", line 795, in train
    self.trainer = (trainer or self._smart_load("trainer"))(overrides=args, _callbacks=self.callbacks)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\models\yolo\detect\train.py", line 65, in __init__
    super().__init__(cfg, overrides, _callbacks)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\trainer.py", line 126, in __init__
    self.device = select_device(self.args.device)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\torch_utils.py", line 199, in select_device
    raise ValueError(
```

第二遍训练提示如下错误，经过检查问data.yaml文件配置中文件名输入错误导致：

```
D:\Anaconda3\envs\yolov11\python.exe D:\test\test.py
Ultralytics 8.3.204 Python-3.10.18 torch-2.5.1 CPU (Intel Core Ultra 5 225H)
engine\trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, cl

Traceback (most recent call last):
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\trainer.py", line 634, in get_dataset
    data = check_det_dataset(self.args.data)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\data\utils.py", line 467, in check_det_dataset
    raise FileNotFoundError(m)
FileNotFoundError: Dataset 'data.yaml' images not found, missing path 'D:\tset\images\val'
Note dataset download directory is 'C:\Users\honor\datasets'. You can update this in 'C:\Users\honor\AppData\Roaming\Ultralytics\settings.json'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "D:\test\test.py", line 7, in <module>
    results = model.train()
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\model.py", line 795, in train
    self.trainer = (trainer or self._smart_load("trainer"))(overrides=args, _callbacks=self.callbacks)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\models\yolo\detect\train.py", line 65, in __init__
    super().__init__(cfg, overrides, _callbacks)
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\trainer.py", line 158, in __init__
    self.data = self.get_dataset()
  File "D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\engine\trainer.py", line 638, in get_dataset
    raise RuntimeError(emojis(f"Dataset '{clean_url(self.args.data)}' error ❌ {e}") from e
RuntimeError: Dataset 'data.yaml' error Dataset 'data.yaml' images not found, missing path 'D:\tset\images\val'
Note dataset download directory is 'C:\Users\honor\datasets'. You can update this in 'C:\Users\honor\AppData\Roaming\Ultralytics\settings.json'
```

第三遍训练一直如下错误，经过多次多次排查，最终发现**标签txt文件导出为空**导致，重新使用label studio进行标注后导出文件，为便于排查，本次只使用6张车牌，结果成功导出：

```
运行 test x
ret = um.true_divide(
D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\metrics.py:850: RuntimeWarning: Mean of empty sl
1 = smooth(f1_curve.mean(0), 0.1).argmax() # max F1 index
D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\metrics.py:582: UserWarning: Glyph 27773 (\N{CJ
fig.savefig(plot_fname, dpi=250)
D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\metrics.py:582: UserWarning: Glyph 36710 (\N{CJ
fig.savefig(plot_fname, dpi=250)
D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\metrics.py:582: UserWarning: Glyph 27773 (\N{CJ
fig.savefig(plot_fname, dpi=250)
D:\Anaconda3\envs\yolov11\lib\site-packages\ultralytics\utils\metrics.py:582: UserWarning: Glyph 36710 (\N{CJ
fig.savefig(plot_fname, dpi=250)
all 2 0 0 0 0
WARNING no labels found in detect set, can not compute metrics without labels
Speed: 0.4ms preprocess, 41.5ms inference, 0.0ms loss, 2.1ms postprocess per image
Results saved to D:\test\runs\detect\yolo11n_custom92
mAP50-95: 0.0
```

所有文件修改后进行再次进行训练，成功：

```
运行 test x
Validating D:\test\runs\detect\yolo11n_custom\weights\best.pt...
Ultralytics 8.3.204 Python-3.10.18 torch-2.5.1 CPU (Intel Core Ultra 5 225H)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 8.5it/s 0.1s
all 2 2 0.00333 1 0.995 0.995
Speed: 0.4ms preprocess, 39.8ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to D:\test\runs\detect\yolo11n_custom
Ultralytics 8.3.204 Python-3.10.18 torch-2.5.1 CPU (Intel Core Ultra 5 225H)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access (ping: 0.00.0 ms, read: 643.3283.7 MB/s, size: 62.1 KB)
val: Scanning D:\test\labels\val.cache... 2 images, 0 backgrounds, 0 corrupt: 100% 2/2 0.0s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 10.7it/s 0.1s
all 2 2 0.00333 1 0.995 0.995
Speed: 0.4ms preprocess, 37.8ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to D:\test\runs\detect\yolo11n_custom2
mAP50-95: 0.9949999999999999
进程已结束，退出代码为 0
```

Data.yaml文档代码

```
path: d:/test
train: images/train #训练集
val: images/val #验证集
test: images/test #测试集
nc: 1
names: ['car']
```

test.py程序代码：

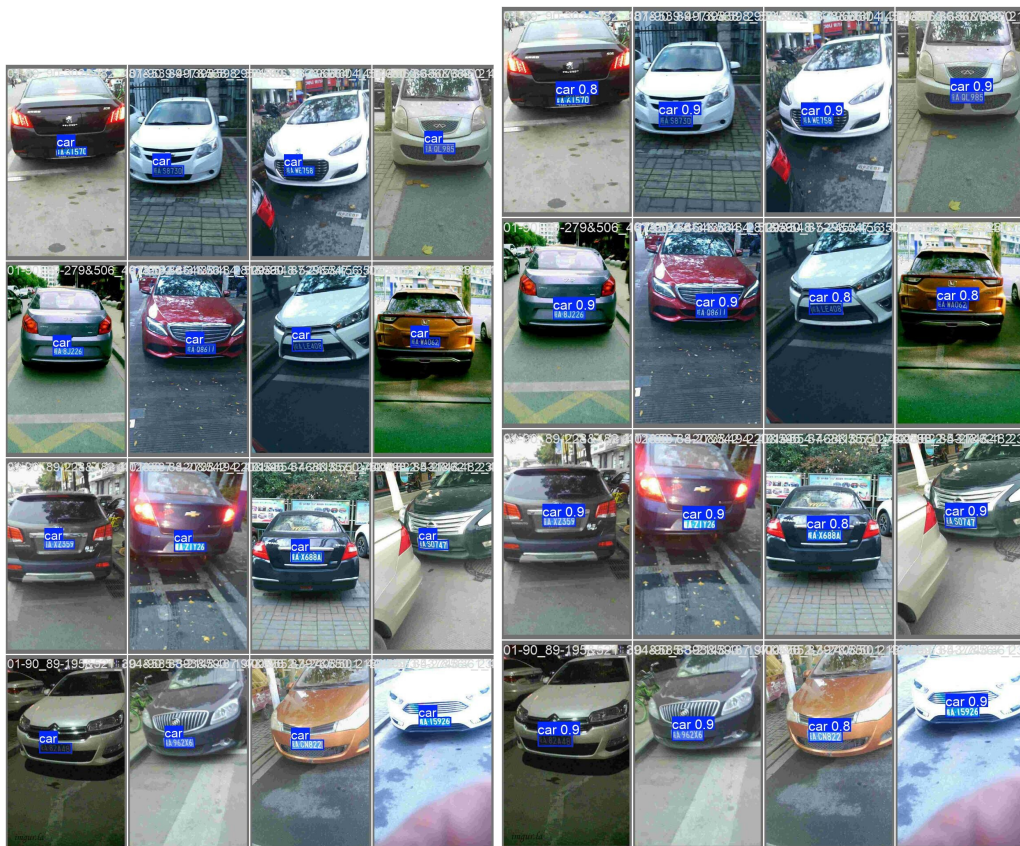
```
from ultralytics import YOLO

# 加载 YOLOv11n 模型
model = YOLO('yolo11n.pt') # 使用预训练模型

# 开始训练
results = model.train(
    data='data.yaml', # 数据集配置文件
    epochs=100, # 训练轮数
    imgsz=640, # 图像尺寸
    batch=16, # 批次大小
    device='cpu', # 设备使用cpu
    name='yolo11n_custom' # 实验名称，结果保存在 runs/detect/yolo11n_custom/
)

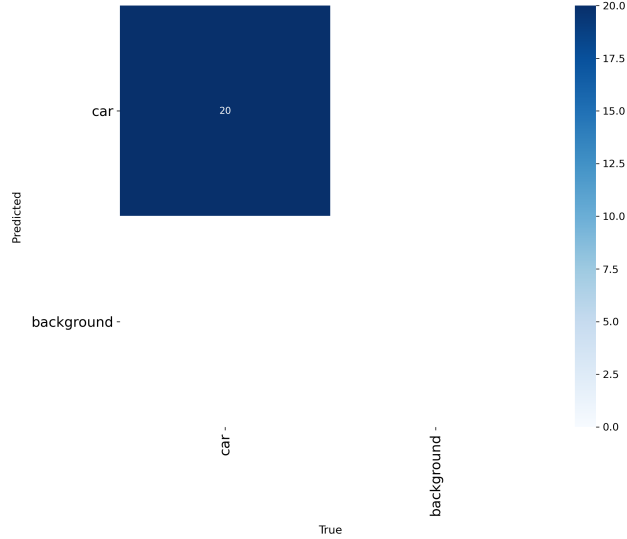
# 可选：评估模型
metrics = model.val()
print("mAP50-95:", metrics.box.map)
```

3.训练结束输出结果:

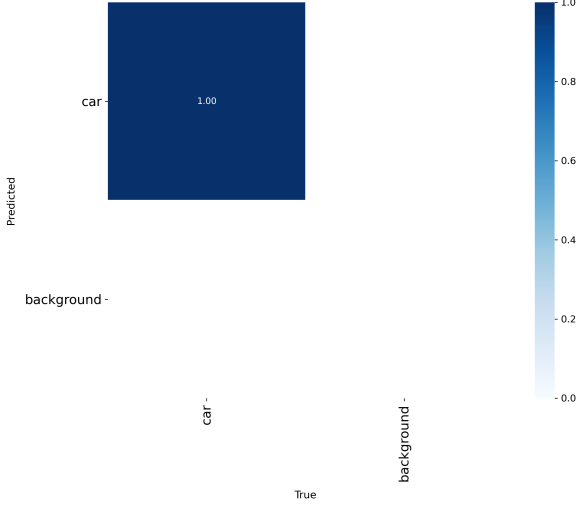


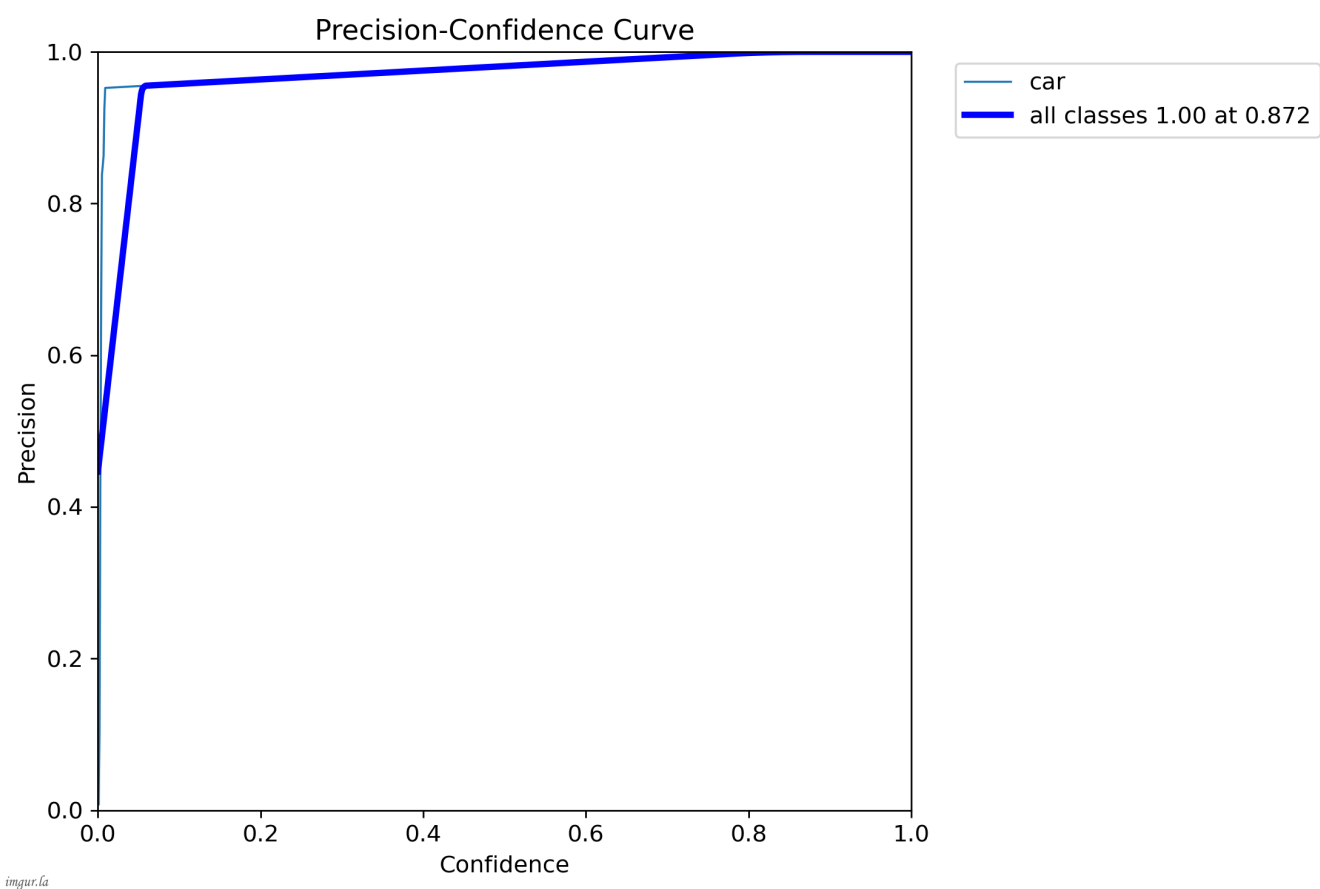
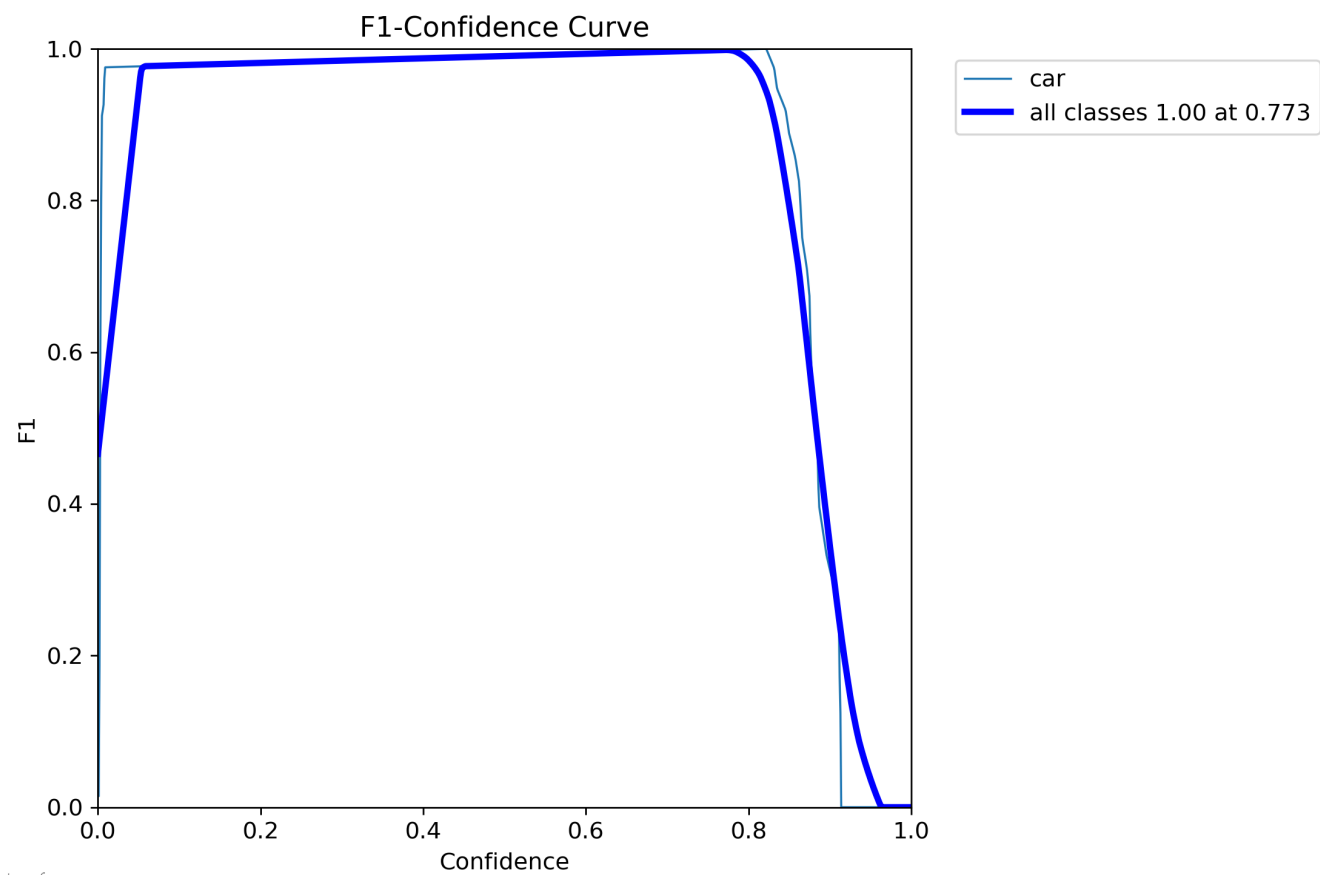


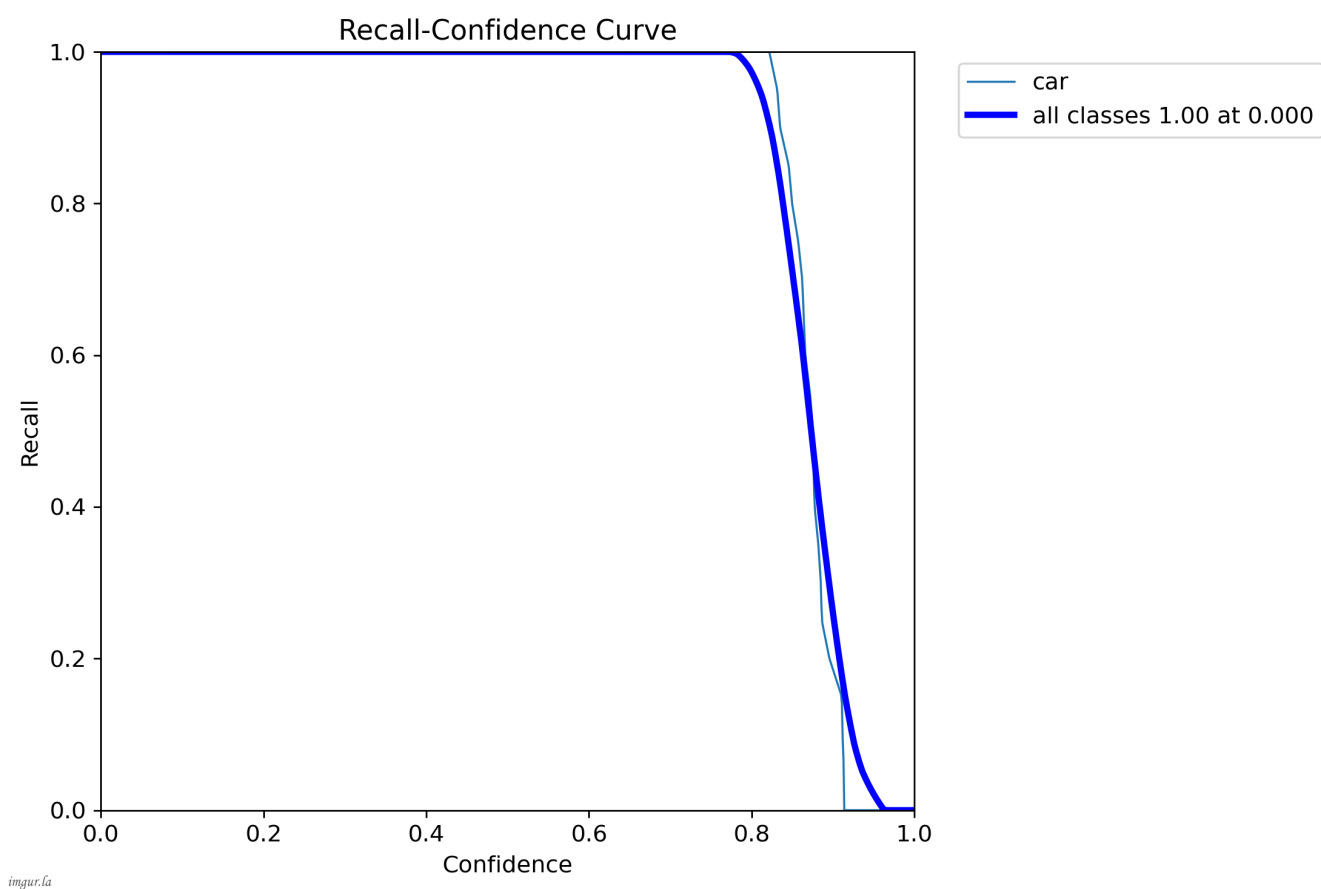
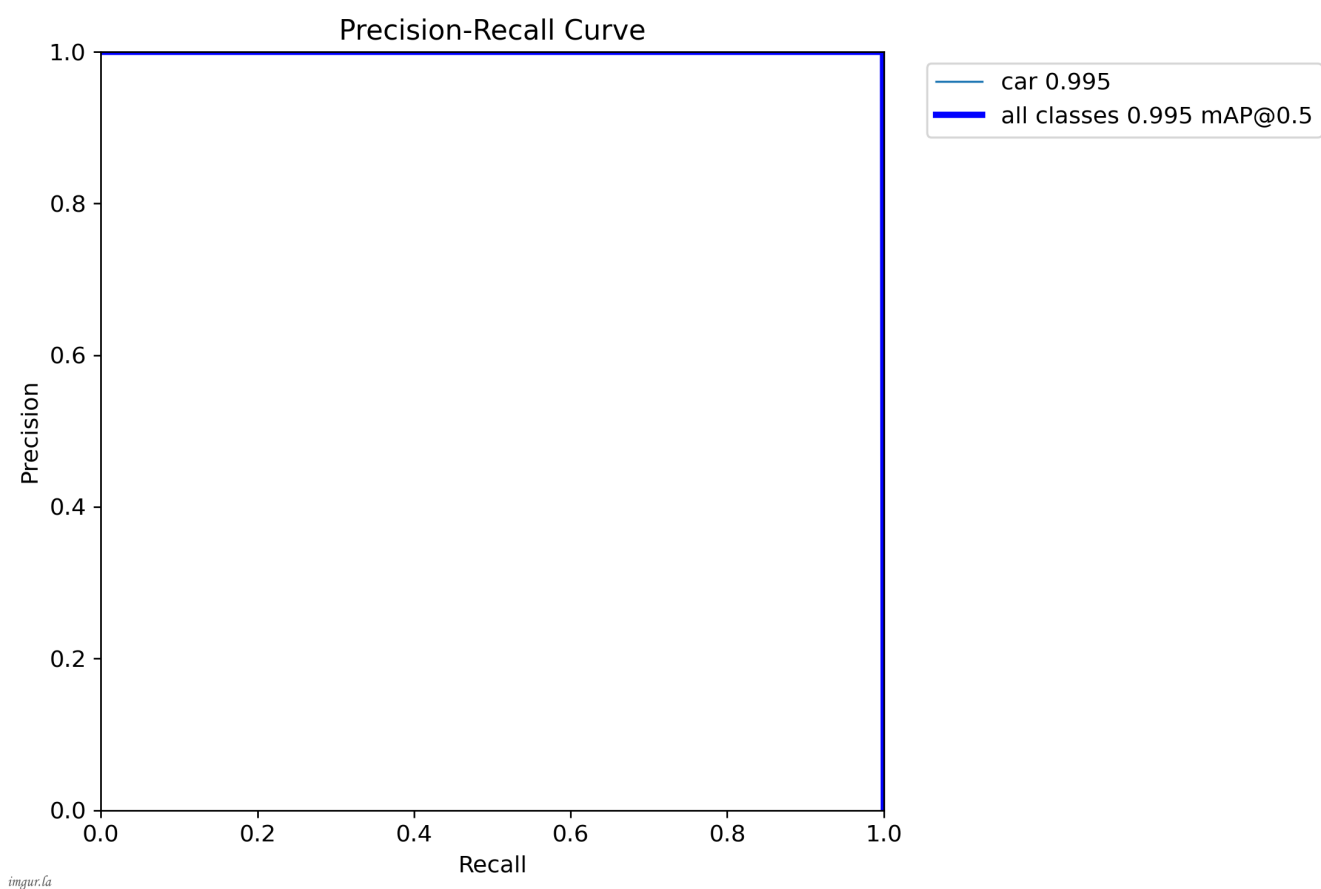
Confusion Matrix



Confusion Matrix Normalized







四、使用训练好的模型进行推理

1. 图片推理

新建picture(hh)程序，代码如下（从文档中复制后修改为实际文件路径等参数）：

```
from ultralytics import YOLO
import cv2
import os
import numpy as np

def diagnose_model():
    """诊断模型问题"""
    model_path = 'd:/hh/runs/detect/yolo11n_custom/weights/best.pt'
    image_path = 'd:/hh/chepai.jpg'

    # 1. 检查文件是否存在
    print("=== 文件检查 ===")
    if not os.path.exists(model_path):
        print(f"❌ 模型文件不存在: {model_path}")
        return
    else:
        print(f"✅ 模型文件存在: {model_path}")

    if not os.path.exists(image_path):
        print(f"❌ 图像文件不存在: {image_path}")
        return
    else:
        print(f"✅ 图像文件存在: {image_path}")

    # 2. 加载模型并检查信息
    print("\n=== 模型信息 ===")
    try:
        model = YOLO(model_path)
        print(f"✅ 模型加载成功")
        print(f"模型类别: {model.names}")
        print(f"类别数量: {len(model.names)}")
    except Exception as e:
        print(f"❌ 模型加载失败: {e}")
        return

    # 3. 检查图像
    print("\n=== 图像信息 ===")
    img = cv2.imread(image_path)
    if img is None:
        print(f"❌ 无法读取图像")
        return
    else:
        print(f"✅ 图像读取成功")
        print(f"图像尺寸: {img.shape}")

    # 4. 测试不同参数进行推理
    print("\n=== 推理测试 ===")
```

```

# 测试1: 使用极低置信度
print("测试1: 极低置信度 (conf=0.01)")
results = model.predict(source=image_path, conf=0.01, save=False)
boxes = results[0].boxes
if boxes is not None and len(boxes) > 0:
    print(f"✅ 检测到 {len(boxes)} 个目标")
    for i, box in enumerate(boxes):
        cls = int(box.cls[0])
        conf = box.conf[0]
        print(f"  目标 {i + 1}: {model.names[cls]} - 置信度 {conf:.3f}")
else:
    print("❌ 未检测到任何目标")

# 测试2: 使用不同图像尺寸
print("\n测试2: 不同图像尺寸")
for imgsz in [320, 640, 1280]:
    results = model.predict(source=image_path, imgsz=imgsz, conf=0.01, save=False)
    boxes = results[0].boxes
    num_detections = len(boxes) if boxes is not None else 0
    print(f"  图像尺寸 {imgsz}: 检测到 {num_detections} 个目标")

# 测试3: 使用保存功能
print("\n测试3: 保存结果")
results = model.predict(source=image_path, conf=0.01, save=True)
print("✅ 推理完成, 结果已保存到 runs/detect/predict 文件夹")

# 5. 手动显示结果
print("\n=== 手动显示结果 ===")
if boxes is not None and len(boxes) > 0:
    annotated_img = results[0].plot()
    cv2.imshow('Detection Results', annotated_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("❌ 没有检测结果可显示")

def check_training_quality():
    """检查训练质量"""
    print("\n=== 训练质量检查 ===")
    model_path = 'd:/hh/runs/detect/yolo11n_custom/weights/best.pt'

    try:
        model = YOLO(model_path)

        # 创建一个简单的测试图像
        test_img = np.ones((640, 640, 3), dtype=np.uint8) * 128
        results = model.predict(source=test_img, conf=0.01, save=False)

```

```

boxes = results[0].boxes
if boxes is not None and len(boxes) > 0:
    print("✅ 模型可以检测目标")
    for box in boxes:
        cls = int(box.cls[0])
        conf = box.conf[0]
        print(f"  检测到: {model.names[cls]} - 置信度: {conf:.3f}")
else:
    print("❌ 模型无法检测任何目标")
    print("可能原因:")
    print("  1. 训练数据不足")
    print("  2. 训练epoch不够")
    print("  3. 模型没有正确学习")
    print("  4. 类别定义有问题")

except Exception as e:
    print(f"检查失败: {e}")

```

```

def alternative_detection_method():
    """备选检测方法"""
    print("\n=== 备选检测方法 ===")
    image_path = 'd:/hh/chepai.jpg'

    img = cv2.imread(image_path)
    if img is None:
        print("无法读取图像")
        return

    # 使用预训练的YOLO模型
    print("尝试使用预训练的YOLOv8模型...")
    pretrained_model = YOLO('yolov8n.pt')

    # 检测车辆
    results = pretrained_model(img, conf=0.3)

    vehicles_detected = 0
    for result in results:
        if result.boxes is not None:
            for box in result.boxes:
                cls = int(box.cls[0])
                class_name = pretrained_model.names[cls]
                if class_name in ['car', 'truck', 'bus']:
                    vehicles_detected += 1
                    print(f"检测到车辆: {class_name}")

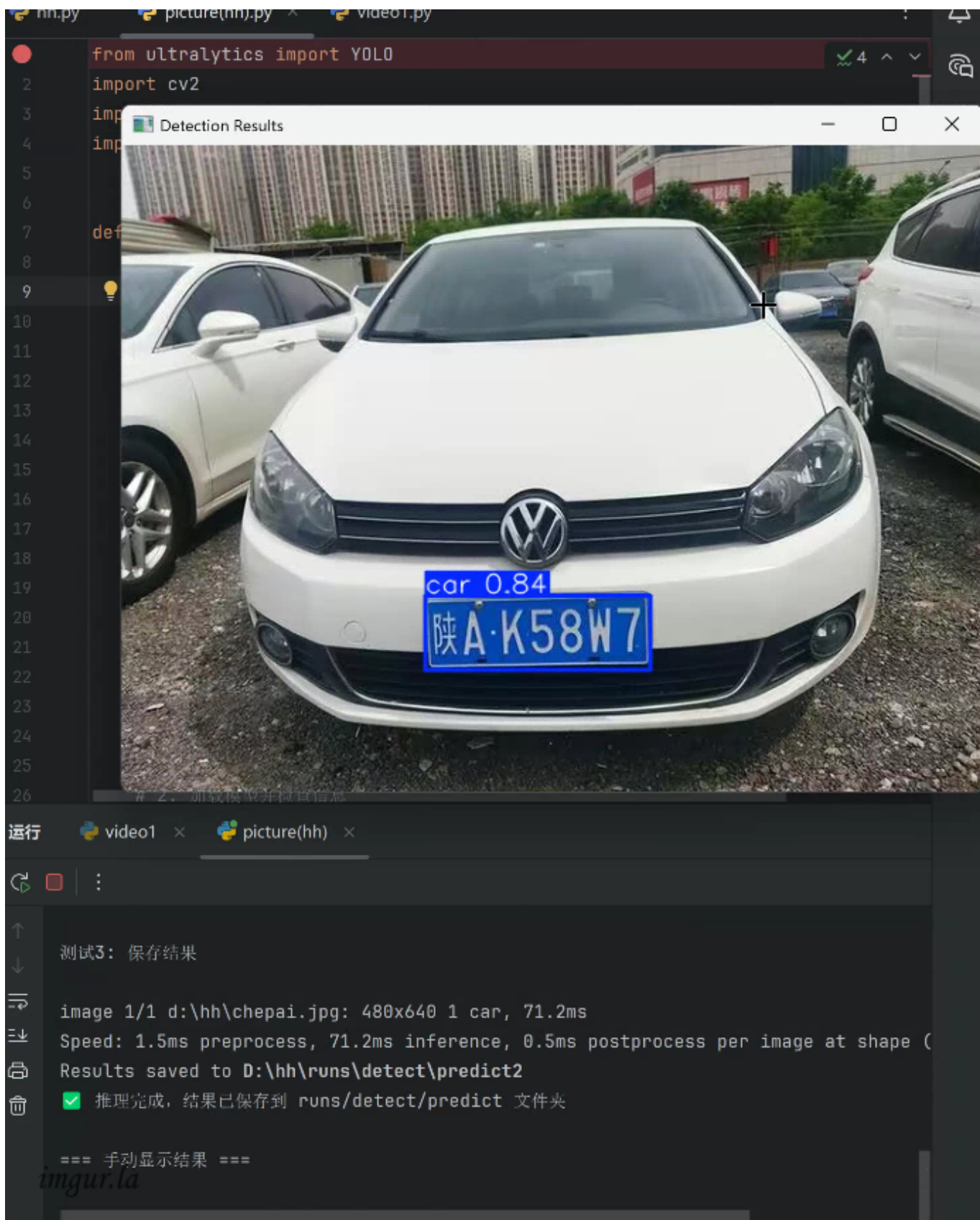
    if vehicles_detected > 0:
        print(f"✅ 预训练模型检测到 {vehicles_detected} 个车辆")

```

```
        annotated_img = results[0].plot()
        cv2.imshow('Pretrained Model Detection', annotated_img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
    else:
        print("❌ 预训练模型也未检测到车辆")
        print("可能图像中确实没有车牌/车辆，或者图像质量有问题")

if __name__ == "__main__":
    print("开始诊断车牌识别问题...")
    diagnose_model()
    check_training_quality()
    alternative_detection_method()
```

显示结果如下，图片成功。



2. 视频推理

新建video1程序，代码如下（从文档中复制后修改为实际文件路径等参数）：

```
import cv2
from ultralytics import YOLO
# 加载模型
model = YOLO('d:/hh/runs/detect/yolo11n_custom/weights/best.pt')
# 视频文件
video_path = "d:/hh/1.mp4"
# 打开视频
cap = cv2.VideoCapture(video_path)
while cap.isOpened():
    # 获取图像
    res, frame = cap.read()
    # 如果读取成功
    if res:
        # 正向推理
        results = model(frame)
    # 绘制结果
    annotated_frame = results[0].plot()
    # 显示图像
    cv2.imshow(winname="YOLO11", mat=annotated_frame)
    # 检测到按ESC退出
    if cv2.waitKey(1) == 27:
        break
    else:
        break
# 释放链接
cap.release()
# 销毁所有窗口
cv2.destroyAllWindows()
```

显示结果如下，视频成功

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

运行

↑
↓
↺
↻
⏮
⏭
⏪
⏩
⏹
⏸
⏵
⏴
⏶
⏷
⏸
⏹
⏺
⏻
⏼
⏽
⏾
⏿

00:00:37

马哥记录仪 bilibili



0: 640x384 1 car, 39.4ms
Speed: 1.4ms preprocess, 39.4ms inference, 0.4ms postprocess per image at shape (1, 3, 640, 384)

ights/best.pt')

..._frame)

postprocess per image at shape (1, 3, 640, 384)

postprocess per image at shape (1, 3, 640, 384)

4.02 CRUF UTF-8 4个空行 Python 3.10 00124